ENSF 460 Project 1
Countdown Timer Application

Group 06 – Lecture Section L01

David Burgoin – UCID: 30214909
Moyo Ogunjobi – UCID:
Jacob Plourde – UCID:

Fall 2026

Contents

1 Project Overview **3**
1.1 System Purpose . . . 3
1.2 Hardware and Software Scope . . . 3
1.3 Design Priorities . . . 3

2 Superloop Architecture **3**
2.1 Execution Model . . . 3
2.2 Shared State and Data . . . 4
2.3 Non-Blocking Timing . . . 4
2.4 State Diagram . . . 5
2.5 State Summary . . . 5

3 Push-Button Event Capture **6**
3.1 GPIO and Change Notification Configuration . . . 6
3.2 Interrupt Service Routine . . . 6
3.3 Combination and Bounce Handling . . . 7

4 Preset Configuration **7**
4.1 Preset Representation and Limits . . . 7
4.2 Incrementing Minutes and Seconds . . . 7
4.3 Decrementing Minutes and Seconds . . . 8
4.4 Preset Display Selection . . . 8

5 Starting and Resetting the Countdown **9**
5.1 Dual-Purpose PB1 and PB2 Command . . . 9
5.2 Short Press: Start . . . 9
5.3 Long Press: Reset . . . 9

6 Running Countdown **9**
6.1 Run Entry and Resume Setup . . . 9
6.2 One-Second Countdown Interval . . . 9
6.3 LED and Terminal Updates . . . 10
6.4 Run Exit Decisions . . . 10

7 Pause, Resume, and Long-Press Clear **10**
7.1 Entering and Holding Pause . . . 10
7.2 Resume or Clear Decision . . . 10
7.3 Preserving a Partial Second . . . 11
7.4 Clear Sequence . . . 11

8 Alarm Behavior **11**
8.1 Alarm Entry . . . 11
8.2 Alarm Indication . . . 11
8.3 Alarm Dismissal Choice . . . 12

9 Terminal Message Selection **12**
9.1 State-to-Message Separation . . . 12
9.2 Single-Line Formatting . . . 12
9.3 UART Duty Cycling . . . 12

10 Corner Cases **13**
10.1 Numeric Boundaries . . . 13
10.2 Button and State Interactions . . . 13
10.3 Timing and Recovery . . . 13

11 UART **14**

12 Timer **14**

13 Power Management **14**
13.1 Low-Power Features in the Test Firmware . . . 14
13.2 Power-Test Methodology . . . 14
13.3 Results and Observations . . . 15
13.4 Power Trace Comparisons . . . 15
13.5 Clock and Peripheral Management in the Test Firmware . . . 21
13.6 Idle and Sleep Operation in the Test Firmware . . . 21
13.7 Timer and UART Scaling in the Test Firmware . . . 21
13.8 Power Management in the Final Design . . . 21

14 Conclusion **22**

1 Project Overview

1.1 System Purpose

The project implements a countdown timer controlled by three push buttons and displayed on a PC terminal. The user can configure minutes and seconds, start the countdown, pause and resume it, clear it with a long press, and observe an alarm when the count reaches zero. The controller is implemented as a finite-state machine in `src/superloop.c`. This report documents the superloop and its state-dependent behavior.

1.2 Hardware and Software Scope

The application targets the PIC24F16KA101. PB1, PB2, and PB3 provide all user input, LED1 indicates a running countdown, and LED2 provides the rapid alarm indication. UART2 produces the required single-line terminal interface. The Change Notification peripheral captures button edges, while Timer2 and Timer3 provide a shared time base for non-blocking delays. The implementation is divided into the superloop, clock configuration, timer, delay, and UART modules so that peripheral-specific code remains separate from the application state machine.

1.3 Design Priorities

The design was guided by four priorities: deterministic button behavior, real-time countdown accuracy, low-power waiting, and short interrupt service routines. The superloop performs all formatting and state decisions, while the ISRs only capture events or update time. Long operations are expressed as states and elapsed-time checks rather than blocking delay loops.

2 Superloop Architecture

2.1 Execution Model

The application repeatedly calls `superloop_loop()` from the infinite loop in `main()`. The superloop separates the active state, stored in `state`, from the next requested state, stored in `request_state`. A transition requested during one iteration is applied at the beginning of the next iteration:

```
if (request_state != state)
{
    prev_state = state;
    state = request_state;
    counter = 0;
}
```

This deferred-transition approach ensures that each state performs one well-defined set of actions per superloop iteration. It also prevents a chain of states from being executed in a single call.

2.2 Shared State and Data

The superloop uses the following shared variables:

Variable	Purpose
`state`	Current finite-state-machine state.
`request_state`	State that will become active on the next iteration.
`buttons`	Push-button bit mask written by the CN interrupt service routine.
`prev_buttons`	Previous button mask used to detect a changed combination.
`counter`	Number of repeated setting operations during a continuous hold.
`preset_mins`	Configured countdown minutes.
`preset_secs`	Configured countdown seconds.
`elapsed_value`	Remaining countdown time represented in seconds.
`elapsed_before_pause`	Partial one-second interval saved when pausing.
`selected_message`	Terminal message selected by the active state.
`delay`	Shared non-blocking delay object used by the state machine.

2.3 Non-Blocking Timing

All user-interface and countdown intervals use one shared `delay_s` object. Calling `delay_start()` records the current value returned by `millis()`, and `delay_elapsed()` later reports completion without stopping the CPU in a software delay loop. The delay implementation carries timing overshoot into the next period instead of restarting from the late observation time. This choice limits accumulated drift when an interval is detected slightly after its nominal boundary.

Only one delay object is required because each stable state owns at most one active timing decision. Entry and transition states reinitialize it when its purpose changes, such as moving from the 500 ms preset-repeat interval to the 3000 ms long-press interval.

2.4 State Diagram

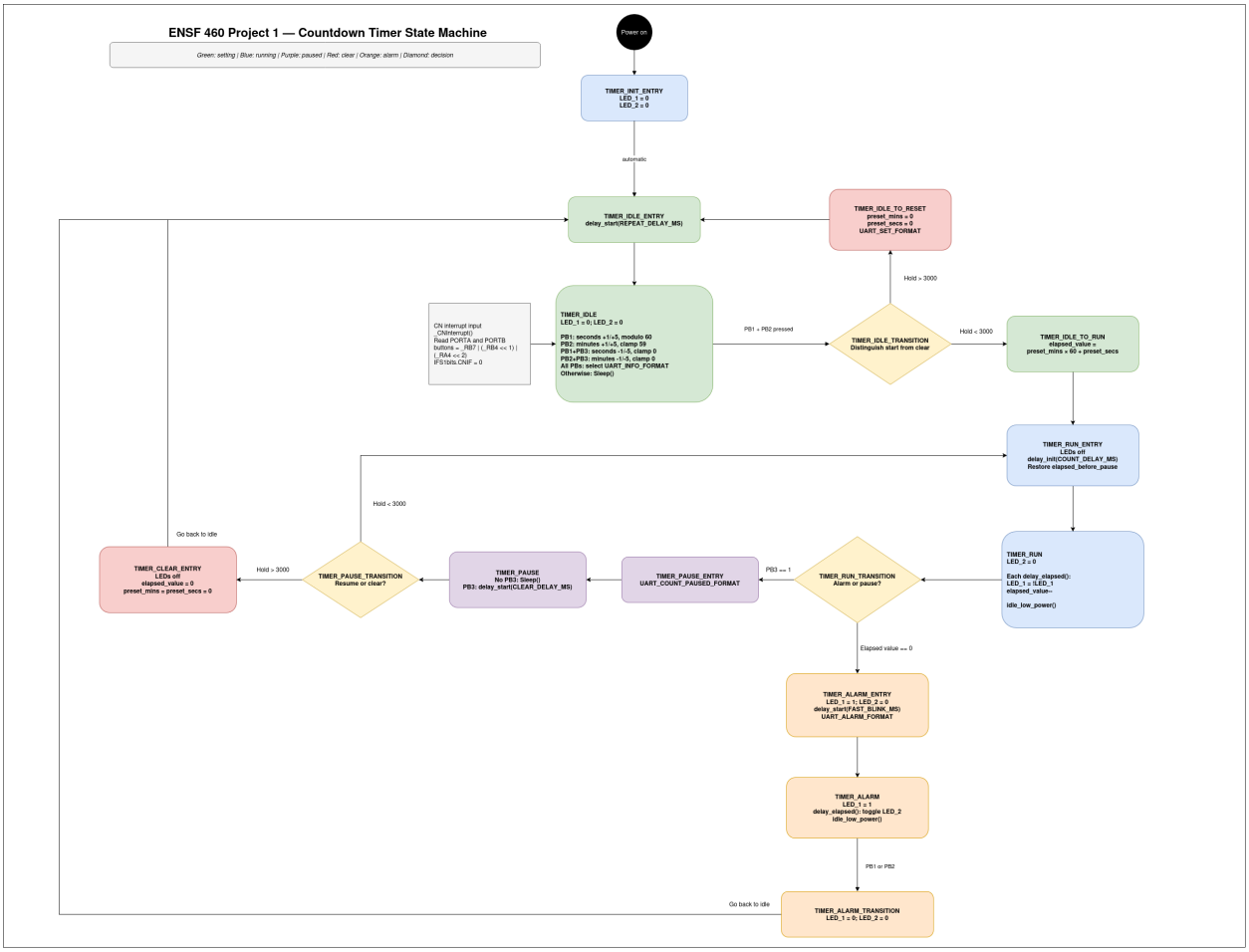


Figure 1: Countdown timer finite-state machine.

2.5 State Summary

The implemented controller contains 17 active states. Entry states perform one-time setup, stable states perform repeated behavior, and transition states decide which state should execute next.

State	Responsibility
`TIMER_INIT_ENTRY`	Turns off both LEDs and requests idle initialization.
`TIMER_IDLE_ENTRY`	Starts the setting-repeat interval before entering idle.
`TIMER_IDLE`	Processes setting controls and the three-button information command.
`TIMER_IDLE_TRANSITION`	Distinguishes a short start command from a held preset-clear command.
`TIMER_IDLE_TO_RUN`	Converts the minute and second preset into total countdown seconds.
`TIMER_IDLE_TO_RESET`	Clears the configured minutes and seconds.

State	Responsibility
TIMER_RUN_ENTRY	Initializes the one-second interval and restores a saved partial interval.
TIMER_RUN	Decrements the timer, updates LED1, and detects completion or PB3.
TIMER_RUN_TRANSITION	Selects the alarm, pause, or clear path.
TIMER_PAUSE_ENTRY	Selects the paused countdown message.
TIMER_PAUSE	Holds the countdown and waits for another PB3 press.
TIMER_PAUSE_TRANSITION	Distinguishes resume from a long-press clear.
TIMER_CLEAR_ENTRY	Clears the countdown, preset, and LED outputs.
TIMER_CLEAR	Selects the clear message before returning to setting mode.
TIMER_ALARM_ENTRY	Enables the alarm indicators and selects the alarm message.
TIMER_ALARM	Holds LED1 on, blinks LED2, and waits for a setting input.
TIMER_ALARM_TRANSITION	Turns off both LEDs and returns to setting mode.

3 Push-Button Event Capture

3.1 GPIO and Change Notification Configuration

The three button pins are configured as digital inputs with internal pull-down resistors. Change Notification is enabled on each input so both press and release edges can wake the processor. In the software bit mask, PB1 occupies bit 0, PB2 bit 1, and PB3 bit 2. This encoding makes individual and simultaneous button tests inexpensive and allows combinations to be represented by values from `0x0` through `0x7`.

3.2 Interrupt Service Routine

The push buttons are handled through the Change Notification peripheral rather than repeated reads of `PORTA` or `PORTB` in the superloop. The CN interrupt service routine first reads both ports to clear the hardware mismatch condition. It then packs the three button inputs into the shared `buttons` bit mask and clears `IFS1bits.CNIF`. (https://ww1.microchip.com/downloads/en/DeviceDoc/39711a.pdf section 12.6.2)

```
void __attribute__((interrupt, no_auto_psv)) _CNInterrupt(void)
{
    volatile uint16_t dummy_a = PORTA;
    volatile uint16_t dummy_b = PORTB;

    buttons = (uint16_t)_RB7
            | ((uint16_t)_RB4 << 1)
            | ((uint16_t)_RA4 << 2);

    (void)dummy_a;
    (void)dummy_b;
    IFS1bits.CNIF = 0;
}
```

The interrupt routine contains no delay, UART output, loop, or function call. All state decisions are deferred to `superloop_loop()`. The `volatile` qualifier on `buttons` informs the compiler that

its value may change asynchronously.

3.3 Combination and Bounce Handling

The internal button enumeration represents every possible three-bit combination. This permits the idle state to select behavior with one `switch` statement instead of repeatedly reading hardware registers.

Mechanical bounce can cause several CN interrupts, but it does not directly modify the timer preset because the ISR only replaces the current bit mask. When the superloop observes a changed combination, it resets the held-button `counter`. Preset changes also require the 500 ms delay to elapse, and long-press commands require a continuous 3000 ms interval. These state-dependent time requirements prevent a cluster of bounce edges from producing a sequence of rapid adjustments. The choice keeps the ISR minimal while placing input interpretation in one testable location.

4 Preset Configuration

4.1 Preset Representation and Limits

The preset is stored as separate signed minute and second values because the setting interface edits the two fields independently. Seconds are maintained in the range 0–59 and minutes in the range 0–59, giving a maximum countdown of 3599 seconds. Signed variables make the decrement paths straightforward; each result is clamped before it can be displayed or converted to the unsigned running value.

4.2 Incrementing Minutes and Seconds

While the timer is idle, holding PB1 increments `preset_secs`, and holding PB2 increments `preset_mins`. An adjustment occurs whenever `delay_elapsed()` reports that the 500 ms repeat interval has completed. The setting therefore changes at a pace that permits the user to stop at a desired value.

The `counter` variable is incremented after every repeated adjustment. For the first ten adjustments, the value changes by one. After `INCREASE_INCREMENT_THRESHOLD` is exceeded, each adjustment changes the value by five:

```
counter++;
preset_secs += counter > INCREASE_INCREMENT_THRESHOLD ? 5 : 1;
preset_secs %= 60;
```

The seconds value uses modulo 60 so a fast increment preserves its remainder when crossing 59. For example, a five-second increment from 58 produces 03. Minutes also support the fast increment, but are clamped at the maximum value of 59.

Whenever the button combination changes or the state machine enters a new state, `counter` is reset to zero. A new continuous hold therefore begins with single-unit adjustments before acceleration is enabled.

The ten-adjustment threshold was selected as a usability compromise. It provides enough slow, single-unit steps for precise setting, then reduces the time required to reach a large preset. Applying the same acceleration rule to minutes and seconds gives the interface consistent behavior.

4.3 Decrementing Minutes and Seconds

PB3 acts as a decrement modifier while setting the timer. Holding PB1 with PB3 decreases seconds, while holding PB2 with PB3 decreases minutes. These combinations use the same repeat interval and the same transition from steps of one to steps of five. Both values are clamped at zero to prevent a negative preset.

```
preset_secs -= counter > INCREASE_INCREMENT_THRESHOLD ? 5 : 1;
preset_secs = preset_secs < 0 ? 0 : preset_secs;
```

PB3 by itself is handled by the idle state's default path and does not modify the preset.

4.4 Preset Display Selection

Every successful preset adjustment selects `UART_SET_FORMAT`. The message is transmitted after state processing, keeping formatted output outside the interrupt service routine. Holding all three push buttons selects `UART_INFO_FORMAT`, which contains the course section and group number.

When no setting command is active, the controller uses the low-power Idle path. After returning from Idle, the repeat delay is restarted so time spent waiting without a setting button does not cause an immediate adjustment on the next press. This preserves the intended 500 ms hold behavior while still allowing the CPU clock to be reduced between events.

5 Starting and Resetting the Countdown

5.1 Dual-Purpose PB1 and PB2 Command

PB1 and PB2 together are used for both starting and clearing the preset. When this combination is detected in `TIMER_IDLE`, the controller starts the 3000 ms clear interval and requests `TIMER_IDLE_TRANSITION`.

Using the duration of one button combination for two commands avoids consuming PB3, which is reserved for decrementing, pausing, resuming, and clearing a running timer. A transition state is used because the controller cannot classify the command until either the buttons are released or the long-press interval completes.

5.2 Short Press: Start

If PB1 and PB2 are released before the interval expires and the preset is nonzero, the controller requests `TIMER_IDLE_TO_RUN`. This state converts the preset into one total-second value:

```
elapsed_value = (uint16_t)(preset_mins * 60 + preset_secs);
```

The conversion is delayed until the start command is accepted. This keeps minute and second editing simple while allowing the running state to perform one comparison and one decrement on a single total-second value.

5.3 Long Press: Reset

If the combination remains held for the complete interval, `TIMER_IDLE_TO_RESET` clears both preset fields and returns to idle setup. If the buttons are released while the preset is 00:00, the controller returns to idle without starting a countdown. This explicitly handles the zero-length timer corner case.

The 3000 ms threshold is long enough to distinguish an intentional reset from the normal two-button start gesture. Resetting through a separate state also ensures the preset is changed once, followed by one deliberate terminal update.

6 Running Countdown

6.1 Run Entry and Resume Setup

`TIMER_RUN_ENTRY` turns off both LEDs, configures the shared delay for the one-second countdown interval, and applies any partial interval saved before a pause. It then requests `TIMER_RUN`.

The same entry state is used for a new countdown and for resuming a paused countdown. For a new run, `elapsed_before_pause` is zero. For a resume, it contains the portion of the interrupted second that had already elapsed. Sharing the entry logic avoids separate countdown implementations with slightly different timing behavior.

6.2 One-Second Countdown Interval

While running, every completed interval toggles LED1 through the `LED_1` macro, which maps to `_LATB9`. If the remaining value is greater than zero, it is decremented exactly once:

```
if (delay_elapsed(&delay))
{
    LED_1 = !LED_1;
    if (elapsed_value > 0)
    {
        elapsed_value--;
    }
}
```

The timer ISRs advance `millis()` in 125 ms increments, and all application intervals are exact multiples of 125 ms. The delay abstraction preserves overshoot, while the timer driver transfers the partial timer phase when switching clocks. These choices prevent repeated Idle transitions from discarding fractions of the one-second countdown period.

6.3 LED and Terminal Updates

The superloop compares `elapsed_value` with `prev_elapsed_value`. A difference selects `UART_COUNT_FORMAT`, ensuring that the terminal count is updated when the displayed second changes rather than on every superloop iteration.

LED1 is toggled by the same one-second event that decrements the count. Coupling the visible indicator to the countdown event guarantees the required one-second on/off cadence without another timer object. LED2 is explicitly held off throughout the running state so an earlier alarm indication cannot remain active.

6.4 Run Exit Decisions

When `elapsed_value` reaches zero, or when the PB3 bit becomes active, the running state saves the time already spent in the current one-second interval and requests `TIMER_RUN_TRANSITION`. The transition state sends a completed countdown to the alarm, a released PB3 to pause, or a PB3 held for 3000 ms to clear.

Countdown completion is tested before the PB3 decision in the transition state. Therefore, reaching zero always produces the alarm even if PB3 changes at the same time. This priority prevents a completed countdown from being silently converted into a pause or clear operation.

7 Pause, Resume, and Long-Press Clear

7.1 Entering and Holding Pause

A short PB3 press while running leads to `TIMER_PAUSE_ENTRY`. This state selects the paused count message and then requests `TIMER_PAUSE`. The countdown value is not decremented while paused.

While paused, the final design continues to use the low-frequency Idle path because Sleep mode is not permitted. Timer interrupts may wake the processor, but the paused state performs no countdown decrement. Only another PB3 command changes the state.

7.2 Resume or Clear Decision

Pressing PB3 again starts a new 3000 ms decision interval in `TIMER_PAUSE_TRANSITION`. Releasing PB3 before the interval expires requests `TIMER_RUN_ENTRY`, which resumes the countdown. Holding PB3 until the interval expires requests `TIMER_CLEAR_ENTRY`.

The same PB3 duration rule is used while running and paused: a release before three seconds is a normal control action, while a continuous three-second hold is a destructive clear. Reusing the threshold makes the interface predictable and avoids a separate reset combination during countdown operation.

7.3 Preserving a Partial Second

The partial interval saved in `elapsed_before_pause` prevents a pause and resume operation from discarding the portion of the current second that had already elapsed. On resume, the delay is enabled with an adjusted start value:

```
delay_enable(&delay, millis() - elapsed_before_pause);
elapsed_before_pause = 0;
```

For example, if PB3 pauses the countdown 375 ms into a second, resuming requires only the remaining 625 ms before the next decrement. Without this correction, repeated pause and resume operations could extend the countdown by almost one second each time.

7.4 Clear Sequence

The clear entry state resets `elapsed_value`, `preset_mins`, and `preset_secs`, turns off both LEDs, and enters `TIMER_CLEAR`. The clear state selects `UART_CLEAR_FORMAT` before returning to idle setup.

Clearing both the active value and the preset prevents a later start from unexpectedly restoring the old duration. Turning off both outputs in the entry state makes the hardware response immediate, while the following state handles the terminal confirmation outside the state that mutates the data.

8 Alarm Behavior

8.1 Alarm Entry

When the countdown reaches zero, `TIMER_ALARM_ENTRY` turns LED1 on, initializes LED2 to off, starts the 250 ms blink interval, and selects `UART_ALARM_FORMAT`. LED1 maps to `_LATB9`, while LED2 maps to `_LATA6`.

Separating alarm entry from the stable alarm state ensures the terminal message and initial LED conditions are established once. LED2 always begins from a known off phase, making the rapid blink pattern repeatable.

8.2 Alarm Indication

In `TIMER_ALARM`, LED1 remains continuously on. Each completed fast-blink interval toggles LED2. Pressing either setting button requests `TIMER_ALARM_TRANSITION`, selects the preset display, turns both LEDs off, and returns the controller to idle setup.

The 250 ms toggle interval produces a complete LED2 blink cycle every 500 ms, visually distinguishing the alarm from LED1's one-second running indication. The state uses the same low-frequency Idle path as the countdown, because timer interrupts must continue to generate the blink events.

8.3 Alarm Dismissal Choice

Either PB1 or PB2 dismisses the alarm because both buttons already represent setting actions. Their press therefore has the intuitive meaning of acknowledging the alarm and returning to preset configuration. PB3 is not used for dismissal, preserving its role as the pause and clear control. The transition state turns off both LEDs before the idle-setting state is entered.

9 Terminal Message Selection

9.1 State-to-Message Separation

The state machine does not print from any interrupt service routine. Instead, each state selects one member of `uart_message_e`. A single `switch` statement at the end of `superloop_loop()` formats the selected message. `UART_NONE` is selected at the start of every iteration so no output is produced unless the state machine explicitly requests it.

This separation allows state code to describe what should be displayed without duplicating UART formatting logic throughout the FSM. It also guarantees that the CN and timer ISRs remain short and never wait for a UART transmitter.

9.2 Single-Line Formatting

The `LINE_CLEAR` prefix uses a carriage return and an ANSI clear-to-end-of-line sequence, so each update replaces the previous terminal contents rather than scrolling. Minutes and seconds are formatted with two digits. A compile-time `DEBUG` option replaces the control sequence with a newline when a scrolling trace is more useful during development.

Selection	Terminal content
`UART_COUNT_FORMAT`	Active countdown value
`UART_COUNT_PAUSED_FORMAT`	Remaining value with a paused indicator
`UART_ALARM_FORMAT`	Finished value and alarm indicator
`UART_SET_FORMAT`	Configured minute and second preset
`UART_INFO_FORMAT`	Course section and group information
`UART_CLEAR_FORMAT`	Cleared 00:00 value

Table 3: Messages selected by the superloop.

9.3 UART Duty Cycling

The `print` macro enables UART2 immediately before formatting and transmission, then calls `uart_disable()` after the bytes have left the transmitter. Disabling waits for `U2STAbits.TRMT`, preventing the peripheral from being turned off in the middle of a message. The UART therefore remains powered only for actual output, and active countdown messages are requested only when `elapsed_value` changes.

10 Corner Cases

10.1 Numeric Boundaries

Starting at 00:00: Releasing PB1 and PB2 with a zero preset returns to idle instead of starting or immediately alarming.

Seconds above 59: Incrementing seconds uses modulo 60, including when a five-second adjustment crosses the boundary.

Minutes above 59: The minute preset is clamped at 59.

Decrement below zero: Both minute and second decrement paths clamp their results at zero.

Countdown underflow: The running state checks `elapsed_value > 0` before decrementing.

10.2 Button and State Interactions

Pause during a partial second: The elapsed portion of the current interval is stored and restored when the countdown resumes.

Arbitrary button combinations: The complete three-bit button enumeration gives each recognized combination an explicit setting behavior or routes it through the default path.

Alarm dismissal: PB1 or PB2 turns off both alarm LEDs and restores the preset display.

Completion and PB3 together: Countdown completion has priority, so the controller enters the alarm rather than losing the completion event.

New setting press after Idle: The repeat delay is restarted after an input-waiting Idle interval, preventing accumulated idle time from causing an immediate repeated adjustment.

10.3 Timing and Recovery

Invalid state value: The default state-machine branch requests `TIMER_INIT_ENTRY`, allowing the controller to return to a known state.

Timer counter rollover: Elapsed time is calculated with unsigned subtraction, so a wrap of the 32-bit millisecond counter does not invalidate normal intervals that are much shorter than the counter range.

Late interval observation: `delay_elapsed()` retains overshoot when scheduling the next interval, preventing repeated superloop or clock-switch latency from accumulating as long-term countdown drift.

Clock switching: The timer driver converts the partial Timer2 or Timer3 phase when changing between 500 kHz and 8 MHz, preserving elapsed time across low-power Idle periods.

11 UART

12 Timer

13 Power Management

13.1 Low-Power Features in the Test Firmware

Several complementary power-saving techniques were added to the test firmware before the measurements were collected. Unused peripheral modules were disabled through the Peripheral Module Disable registers. Push-button activity and elapsed time were detected with Change Notification and timer interrupts so the processor did not need to poll the input ports or timer registers. When the processor was waiting, the test firmware entered either `Idle()` or `Sleep()` instead of executing a busy loop. Before entering these modes, UART2 was allowed to finish its current transmission and was then shut down. The UART was only enabled again when a new message had to be sent, and messages were suppressed when the displayed value had not changed. The test firmware also evaluated dynamic clock scaling and reconfigured its timer and UART drivers when the system clock changed. Together, peripheral gating, interrupt-driven operation, low-power CPU modes, dynamic clock scaling, UART duty cycling, and output-on-change messaging reduced the time for which the CPU and peripherals remained fully active.

13.2 Power-Test Methodology

Power consumption was investigated by operating the microcontroller at several clock frequencies and supply voltages. The tested clock settings were 31 kHz, 500 kHz, 1 MHz, 4 MHz, 8 MHz, and 32 MHz. The lowest-frequency setting was rounded to 32 kHz in the firmware constants. Each clock setting was tested at supply voltages of 1.8 V, 2.4 V, 3.0 V, and 3.3 V. This produced a test matrix of 24 frequency-and-voltage combinations, with two automated usage profiles attempted for each combination. The purpose of these tests was to determine how far the clock frequency and supply voltage could be reduced while preserving correct timer, push-button, LED, and UART operation. The voltage was adjusted externally and was not selected by the firmware.

The two profiles were scripts from the hardware-in-the-loop test harness. Test 03, `power_short_run_alarm_dwell`, was the more active workload. It configured and started two short countdowns, allowed each countdown to finish, remained in the alarm state for extended periods, and dismissed the alarms using the setting buttons. This profile repeatedly exercised countdown timing, UART updates, LED1 blinking, and the rapid LED2 alarm blink, with only brief idle intervals between operations.

Test 04, `power_idle_preset_fiddling`, represented a mostly idle or sleeping workload. It did not start the countdown. Instead, it left the controller in the preset-setting state, separated short button operations with waiting intervals, and adjusted the seconds and minutes in both directions. The profile also held each setting combination for ten seconds to exercise button auto-repeat and accelerated adjustment. Between button events the processor could enter `Sleep()`, while during a held button it repeatedly returned to the low-frequency `Idle()` path. Test 04 therefore measured a usage pattern dominated by low-power waiting, with brief wakeups for Change Notification interrupts, preset calculations, and UART output. Table 4 gives the measured average current for both profiles in microamperes.

	1.8 V		2.4 V		3.0 V		3.3 V	
Clock	03	04	03	04	03	04	03	04
31 kHz	X	X	X	X	16.9	24.1	19.3	27.3
500 kHz	16.4	17.4	19.8	20.1	20.7	21.5	24.1	25.7
1 MHz	20.1	21.8	25.9	24.1	28.1	27.2	36.6	30.9
4 MHz	29.4	28.7	30.9	30.2	33.2	34.4	56.4	42.4
8 MHz	34.0*	29.7*	40.6	25.2	79.4	40.3	90.5	57.9
32 MHz	20.9*	18.1*	29.4	26.0	35.3	29.1	57.3	37.6

Table 4: Measured average current in μA for power-profiling Tests 03 and 04. An X indicates that the configuration failed the test. An asterisk marks a measurement taken outside the device's specified operating range.

13.3 Results and Observations

The measurements generally show that reducing the supply voltage and clock frequency reduced average current. The lowest result from a configuration within its specified operating range was 16.4 μA for Test 03 at 500 kHz and 1.8 V. At 3.0 V, where every clock setting passed both profiles, the 31 kHz configuration measured 16.9 μA in Test 03 and 24.1 μA in Test 04. The 31 kHz configuration failed both profiles at 1.8 and 2.4 V. The 8 and 32 MHz measurements at 1.8 V are marked with an asterisk because these clock rates require a minimum supply of approximately 2.4 V. Although current readings were obtained, these four points were outside the specified operating range and were excluded when identifying the lowest valid result.

The profile also affected the average because Test 03 repeatedly ran the countdown and alarm outputs, whereas Test 04 remained in preset mode and spent most of its duration waiting in `Idle()` or `Sleep()`. Consequently, readings from the two test columns describe different workloads and should not be treated as repeat measurements of an identical sequence. The non-monotonic results at the higher clock settings also show that clock frequency alone does not determine the average: completing active work more quickly and returning to a low-power mode can offset some of the higher instantaneous current of a faster clock.

13.4 Power Trace Comparisons

Figures 2–4 show current over time for three pairs of clock settings. All three comparisons used the active Test 03 workload at a 3.3 V supply. Each image shows the two individual captures and an aligned overlay after the first two seconds were removed. These traces make the short current peaks caused by button processing, UART activity, countdown updates, and alarm LED transitions visible in addition to the average values reported in Table 4.

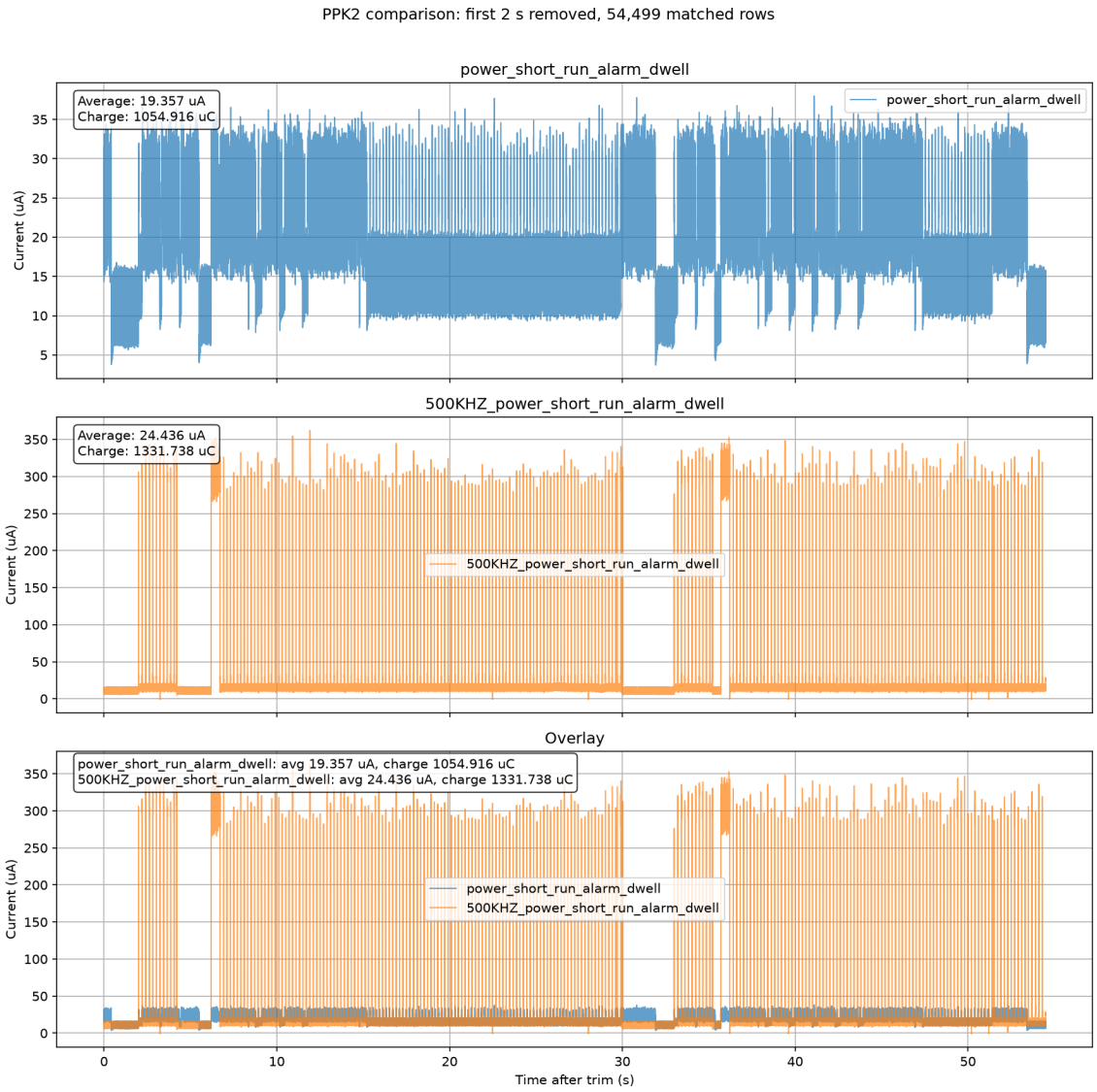


Figure 2: Test 03 at 3.3 V comparing the 31/32 kHz and 500 kHz clock configurations. The measured averages shown on the graph are 19.357 μA and 24.436 μA, respectively.

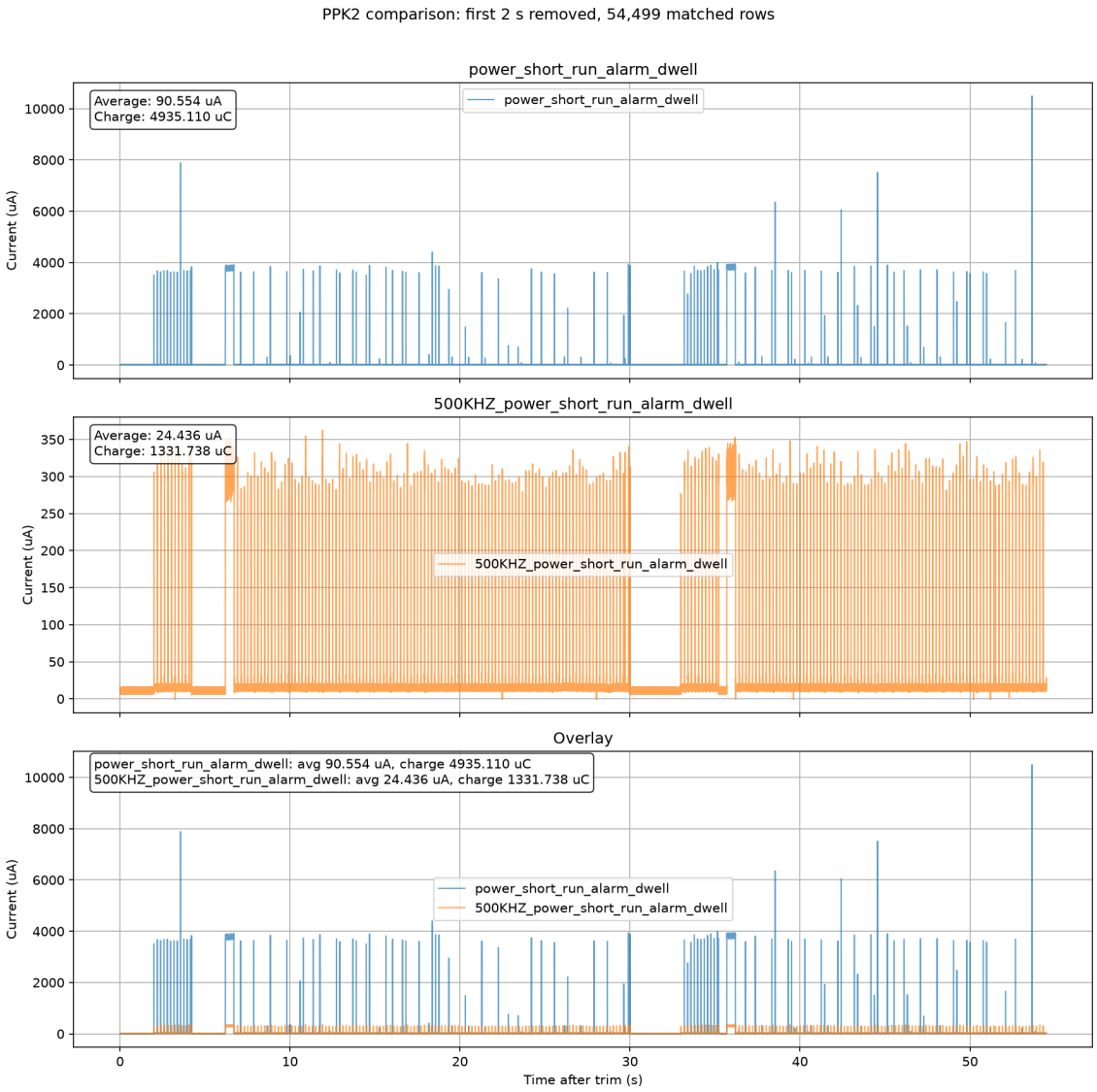


Figure 3: Test 03 at 3.3 V comparing the 8 MHz and 500 kHz clock configurations. The measured averages shown on the graph are 90.554 μA and 24.436 μA, respectively.

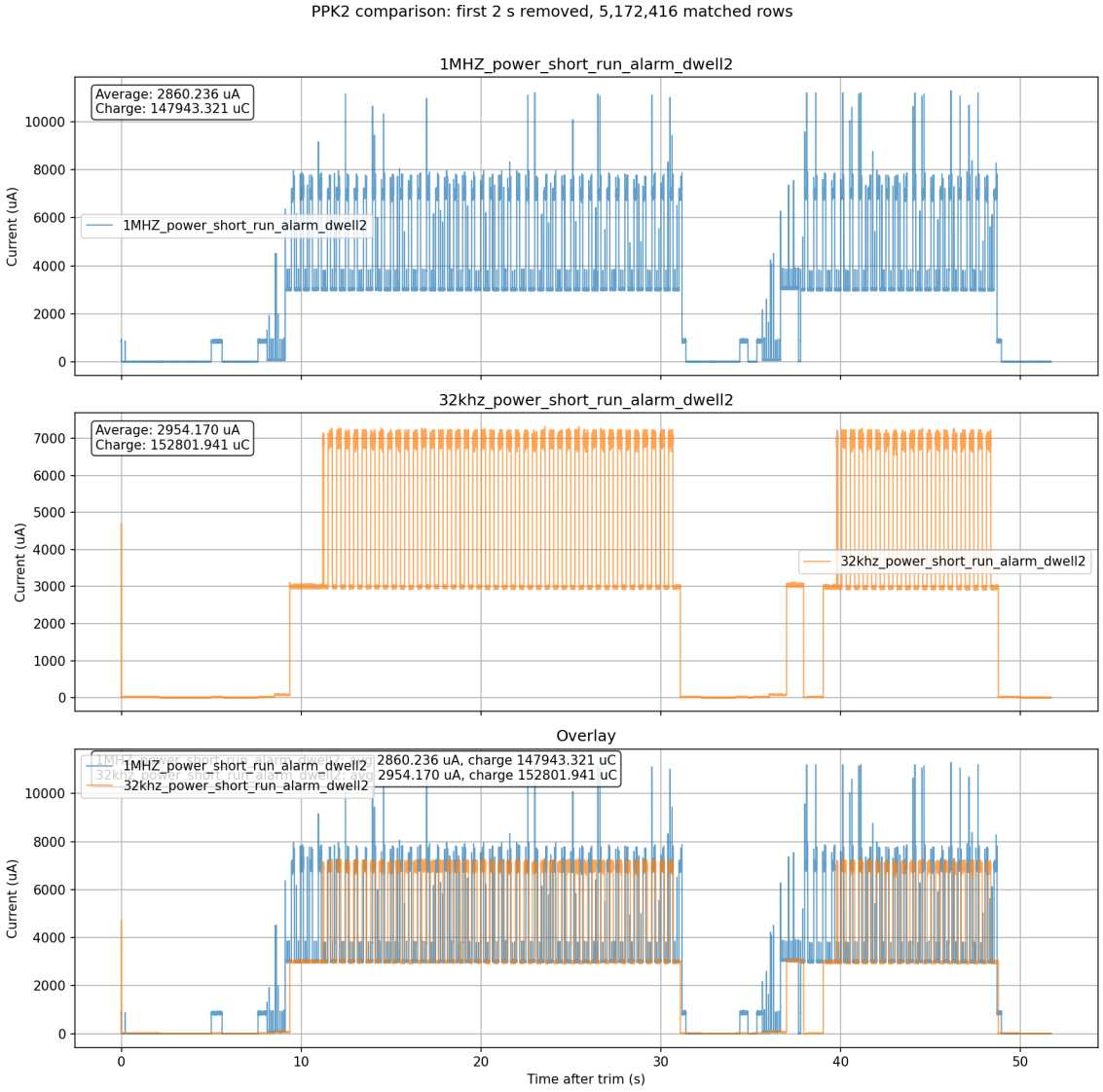


Figure 4: Test 03 at 3.3 V comparing the 1 MHz and 31/32 kHz clock configurations. In this capture the 1 MHz configuration had the lower average, illustrating that the slowest clock does not necessarily minimize total energy for an active workload.

Figure 5 isolates the effect of the hardware outputs and pull-down resistors. With these loads disabled, the 500 kHz Test 03 capture averaged 18.881 μA. Enabling the LEDs and pull-downs increased the measured average to 1928.130 μA, or approximately 1.93 mA. The comparison shows that the external LED load dominates the tens-of-microamperes differences produced by clock selection. The clock and CPU-mode tests were therefore useful for optimizing the microcontroller itself, but the complete final system is expected to consume substantially more current whenever its required LEDs are active.



Figure 5: Test 03 comparison with the LED outputs and pull-down resistors disabled versus enabled. Enabling the hardware loads increased the average current from 18.881 μA to 1928.130 μA.

13.5 Clock and Peripheral Management in the Test Firmware

The profiling firmware supported all six test frequencies and changed the oscillator source or FRC divider to select each operating point. Before a clock change, it waited for any UART transmission to finish, prevented interrupts during the transition, stopped the active timer, performed the oscillator switch, and reconfigured the timer and UART for the new frequency. The Peripheral Module Disable registers were also used to turn off hardware that was not required by the application. These mechanisms allowed the same workload to be repeated at each point in the frequency-and-voltage matrix without changing the intended state-machine behavior.

13.6 Idle and Sleep Operation in the Test Firmware

The test firmware evaluated two low-power paths. Its idle path shut down UART2, selected a low-frequency clock, and executed `Idle()`. An interrupt woke the processor, after which the selected test frequency was restored. Its sleep path waited until UART output was complete, disabled UART2, and executed `Sleep()`. A Change Notification interrupt from a push button could then wake the device. UART2 was re-enabled only when another terminal message had to be sent.

13.7 Timer and UART Scaling in the Test Firmware

Maintaining timing accuracy across the test frequencies required the timer period and prescaler to be selected for the active clock. The profiling firmware chose the appropriate timer source when changing clocks so that the state machine could continue using the same elapsed-time calculations for button-repeat intervals, the three-second long-press interval, the one-second countdown, and the alarm blink interval. The UART baud rate was also reduced at lower clock frequencies and its baud-rate generator was recalculated after a clock change. These changes allowed each frequency to be tested without treating slow UART output or a changed timer tick as part of the intended delay.

13.8 Power Management in the Final Design

The final submitted design uses the profiling results to separate active processing from low-power waiting. Normal state-machine processing and all UART transmissions run from the 8 MHz FRC clock. The project requires the processor to use `Idle()` while waiting and does not permit `Sleep()` in the final design. The `idle_low_power()` macro therefore calls `timer_switch(FREQ_500_KHZ)`, executes `Idle()`, and restores `FREQ_8_MHZ` after an interrupt wakes the processor. The 500 kHz clock was selected as the low-power idle frequency because it passed both automated profiles at every tested supply voltage and produced the lowest valid measured current, 16.4 μA, in the matrix.

The final timer driver maintains a common time base while switching between these two clocks. Timer3 is configured for 8 MHz and Timer2 for 500 kHz, with both producing a 125 ms update interval. Before enabling the new timer, `timer_switch()` converts the partial count from the old timer into the equivalent phase of the new timer. It also includes measured clock-switch overhead corrections. This preserves the countdown phase and avoids accumulating a large timing error every time the processor enters or leaves `Idle()`.

The same low-power Idle path is used when no setting button is active, while the countdown is paused, while the countdown is running, and while the alarm is active. In the preset-setting state, the repeat delay is restarted after returning from Idle so that a newly pressed setting button must still be held for the intended interval before its first repeated adjustment. While paused, the timer continues to advance but the countdown value is not decremented; the saved partial-second value

is applied when the countdown resumes. The three push buttons are captured by the CN interrupt service routine, and elapsed time is updated by the timer ISRs. The superloop therefore reacts to interrupt-produced events instead of continuously polling the input ports or timer registers.

Using Idle instead of Sleep has an unavoidable power cost. In Idle mode the peripheral clocks and the selected timer continue running, so the 125 ms timer interrupt periodically wakes the CPU even in states that only need to wait for a button. Each wake also causes the firmware to return temporarily to 8 MHz before entering the 500 kHz Idle path again. Sleep would stop more of the device and would normally produce a lower average current, but it cannot be used under the final design constraint. The final implementation is therefore expected to have degraded power performance compared with the Sleep-enabled profiling firmware and its measurements. The profiling results remain useful for selecting the lowest practical Idle clock, but they represent a best-case comparison rather than the exact consumption expected from the final firmware.

UART2 is initialized at 500000 baud but is disabled whenever no message is being transmitted. The `print` macro enables UART2, formats and sends one selected message, waits for the transmission to finish through `uart_disable()`, and then turns the interface off again. The state machine defaults to `UART_NONE` on every iteration and requests output only for a state event or displayed-value change. As a result, the UART and its high-speed clock are not kept active merely to redraw an unchanged terminal line. The final design therefore combines 8 MHz active execution, 500 kHz low-power Idle operation, interrupt-driven wakeups, phase-preserving timer switching, and UART duty cycling while accepting the additional consumption caused by the prohibition on Sleep mode.

14 Conclusion

The superloop implements the countdown timer as a finite-state machine with separate entry, stable, and decision states. Button events are captured by the CN interrupt and processed outside the ISR. The state logic covers preset adjustment, accelerated changes, starting, preset reset, countdown operation, pause and resume, long-press clear, alarm behavior, LED control, message selection, and boundary conditions.